LESSON PLAN · SCRATCH (GR 6–8) INTERMEDIATE: DATA & SIMULATIONS

Lesson 11: Robust Apps: Validation and Menus

Module 4 | 45 minutes | Validation and program structure | Grades 6–8 · ages 11–14

Objective: Make apps robust: re-ask until input is valid, handle every menu choice, and organise features into procedures.

Success criteria

- I can validate input with a loop.
- I can build a menu that handles every choice.
- I can organise features into procedures.

Key vocabulary

- **validation**: Checking input before using it.
- **robust**: Keeps working even with unexpected input.
- **catch-all**: An else that handles everything not expected.
- **modular**: Built from separate, focused parts.

You will need

- Projector with Kitabash (lesson video, Code Lab)
- Devices: 1 per learner or pair
- Printed worksheet (optional)

Timed lesson plan (45 min)

Time	Stage	What happens
5 min	**Warm-up (unplugged)**	Bouncer (5 min): learners present “inputs” on cards; the bouncer accepts or rejects by a rule.
5 min	**Watch**	Play the lesson video with captions; pause at each heading and ask a question.
8 min	**Build along**	Robust age checker and menu: learners follow the steps.
14 min	**Perform**	Code Lab: validate input, Code Lab: menu
9 min	**Probe**	Valid or invalid for “Enter a test score from 0 to 100”?, activity
4 min	**Wrap-up**	Exit ticket: Write the condition for a valid percentage (0 to 100).

Unplugged warm-up

Bouncer (5 min): learners present “inputs” on cards; the bouncer accepts or rejects by a rule.

Teaching the idea

Robustness: validation loops (repeat until valid) guard against bad input; menus use exhaustive if-else chains with a catch-all else; features are modularised into procedures. Test with invalid, boundary and empty inputs.

Analogy: A bouncer who keeps checking IDs until a valid one is shown.

Live demo: Break an unvalidated app with “banana”; then add the validation loop.

Questions to ask

Ask	Listen for
When does the validation loop stop?	When the input is valid.
Why have a final else in a menu?	To handle unexpected choices.
Why put options in procedures?	Readable and extendable.

Differentiation

Support
Provide the validation condition.

Stretch
Wrap the menu in a loop so the user can choose again until they pick “quit”.

Common misconceptions

Misconception	How to fix it
Users always follow instructions	Validate everything.
if without else is fine for menus	Handle unknown choices too.

Assessment

Use the Code Lab results (auto-checked), the quick checks and the Probe answers. Written answers arrive in your review queue.

Exit ticket
Write the condition for a valid percentage (0 to 100).

Homework
Try typing nonsense into an app or website form. How does it respond?

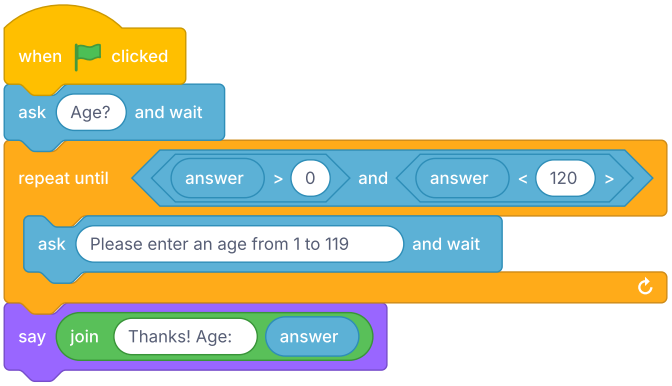
Answer key — Kitabash activities

1. Code Lab: validate input (perform · code lab)

Goals checked: Rejects bad input · Accepts good input · Valid first time

One solution:

Host

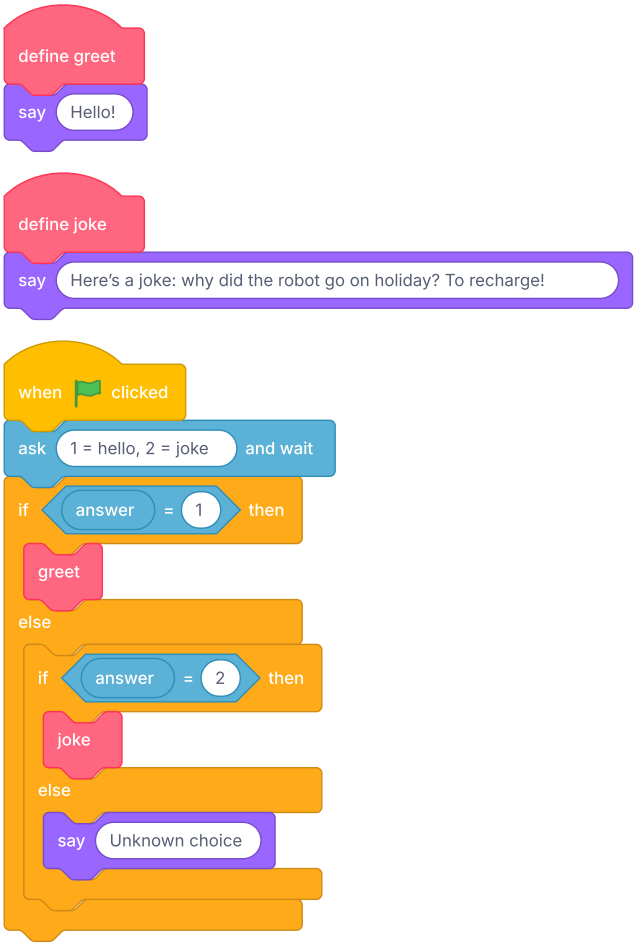


2. Code Lab: menu (perform · code lab)

Goals checked: Option 1 works · Option 2 works · Unknown choices handled · Uses procedures

One solution:

Host



3. Valid or invalid for "Enter a test score from 0 to 100"? (probe · categorize)

Valid: 75, 100, 0
Invalid: -3, seventy, 250

4. Explain your thinking (probe · free response)

Look for: input, invalid, ask. **Example:** 1. Nothing at all: ask again with a hint. 2. A word instead of a number: say "Please type a number" and ask again. 3. A negative number: explain the range. 4. A huge number: say it is too large and ask again.

Answer key — worksheet

1. **Key words:** validation = Checking input before using it. · robust = Keeps working even with unexpected input. · catch-all = An else that handles everything not expected. · modular = Built from separate, focused parts.
2. **Plan your code:** Goals: Rejects bad input; Accepts good input; Valid first time.

3. **Plan your code:** Goals: Option 1 works; Option 2 works; Unknown choices handled; Uses procedures.
4. **Valid or invalid for “Enter a test score from 0 to 100”?:** Valid: 75, 100, 0 · Invalid: -3, seventy, 250
5. **Explain your thinking:** 1. Nothing at all: ask again with a hint. 2. A word instead of a number: say “Please type a number” and ask again. 3. A negative number: explain the range. 4. A huge number: say it is too large and ask again.

Kitabash original curriculum. Scratch is a project of the Scratch Foundation in collaboration with the MIT Media Lab; this course is not affiliated with or endorsed by them.